

Lab 06

DECISION TREE

CODE

```
import math
from collections import Counter

# Calculate Entropy
def entropy(data):
    labels = [row[-1] for row in data]
    counts = Counter(labels)

    ent = 0
    total = len(labels)

    for count in counts.values():
        p = count / total
        ent -= p * math.log2(p)

    return ent

# Information Gain
def information_gain(data, feature_index):
    total_entropy = entropy(data)

    values = set(row[feature_index] for row in data)

    weighted_entropy = 0

    for value in values:
        subset = [row for row in data if row[feature_index] == value]

        weighted_entropy += (len(subset) / len(data)) * entropy(subset)

    return total_entropy - weighted_entropy

# Build Tree
def build_tree(data, features):

    labels = [row[-1] for row in data]

    # All same class
    if len(set(labels)) == 1:
        return labels[0]

    # No features left
    if len(features) == 0:
        return Counter(labels).most_common(1)[0][0]

    gains = [information_gain(data, i) for i in range(len(features))]

    best_feature_index = gains.index(max(gains))
```

```

best_feature = features[best_feature_index]

tree = {best_feature: {}}

values = set(row[best_feature_index] for row in data)

for value in values:
    subset = []

    for row in data:
        if row[best_feature_index] == value:
            new_row = row[:best_feature_index] + row[best_feature_index+1:]
            subset.append(new_row)

    remaining_features = features[:best_feature_index] +
features[best_feature_index+1:]

    tree[best_feature][value] = build_tree(subset, remaining_features)

return tree

# Predict
def predict(tree, features, sample):

    if not isinstance(tree, dict):
        return tree

    root = next(iter(tree))
    feature_index = features.index(root)

    value = sample[feature_index]

    subtree = tree[root][value]

    return predict(subtree, features, sample)

# Dataset (Play Tennis)
data = [
    ["Sunny", "Hot", "No"],
    ["Sunny", "Cool", "No"],
    ["Overcast", "Hot", "Yes"],
    ["Rainy", "Mild", "Yes"],
    ["Rainy", "Cool", "Yes"]
]

features = ["Outlook", "Temperature"]

tree = build_tree(data, features)

print("Decision Tree:")
print(tree)

sample = ["Sunny", "Hot"]

```

```
result = predict(tree, features, sample)
```

```
print("Prediction:", result)
```

OUTPUT

```
● PS E:\class> python -u "e:\class\decisiontree.py"
Decision Tree:
{'Outlook': {'Rainy': 'Yes', 'Overcast': 'Yes', 'Sunny': 'No'}}
Prediction: No
○ PS E:\class>
```